

Experiment No. 4 - Develop a Single Page Application (SPA) using React Router, Hooks, and Component Lifecycle Methods

Aim: To architect and implement an enterprise Single Page Application (SPA) for the Pet Care Management System using React Router DOM v6, lifecycle orchestration hooks (useEffect, useContext), and navigation guards without full-page browser reloads.

Tools Used:

- Visual Studio Code (IDE)
- React 18 & ReactDOM Libraries
- React Router DOM v6 (Declarative Client-Side Routing)
- Google Chrome DevTools (Network & Performance Panels)

Theory:

Single Page Applications (SPAs) represent the standard architectural pattern for modern web applications. Unlike traditional multi-page web applications that request distinct HTML documents from the server on every navigation action, an SPA loads a single root HTML page once. Subsequent navigational transitions dynamically intercept URL path changes and mount appropriate component views entirely within the client runtime.

React Router DOM v6 provides the routing backbone through declarative URL synchronization. Components like `<BrowserRouter>`, `<Routes>`, `<Route>`, and `<Link>` map browser URL routes to specific page components. Dynamic route parameters (`/pets/:id`) capture entity identifiers directly from the address bar via the `useParams()` hook, while programmatic navigation is executed via `useNavigate()`.

Component lifecycle events are orchestrated using the `useEffect` hook, consolidating the responsibilities of legacy class lifecycle methods (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`). `useEffect` manages asynchronous data fetching, subscription setups, and memory leak cleanup through returned cleanup functions. Global authentication state is managed via `useContext`, enabling route protection via specialized wrapper components.

Functions and Methods Used: `createBrowserRouter()`, `<Routes>`, `<Route>`, `useNavigate()`, `useParams()`, `useLocation()`, `useEffect()`, and `useContext()`.

Methodology:

The navigation architecture defines a centralized route registry in `App.jsx`. Public routes (`/`, `/login`, `/register`) are accessible to all visitors, while sensitive operational views (`/owner-`

dashboard`, `/pets`, `/book-appointment`, `/admin-dashboard`) are guarded by an authentication guard component (`ProtectedRoute.jsx`).

Data synchronization relies on `useEffect` hooks embedded within each view to fetch real-time records upon route activation. Visual loading skeletons maintain user engagement while asynchronous data fetching completes.

Procedure:

1. Install React Router DOM by executing `npm install react-router-dom` within the React project.
2. Wrap the root ```` component with ```` inside `src/main.jsx`.
3. Define top-level routes inside `App.jsx` using ```` and nested ```` declarations.
4. Construct a persistent responsive ```` using ```` elements with active tab highlighting.
5. Create `ProtectedRoute.jsx` checking authentication tokens and redirecting unauthorized visitors to `/login`.
6. Implement parameterized routing for pet profile inspection using `` } />`.
7. Inside `PetDetails.jsx`, extract the active ID parameter using `const { id } = useParams()`.
8. Employ `useEffect` to fetch corresponding pet diagnostic records when the route parameter changes.
9. Implement a catch-all 404 route `` } />` to handle undefined paths gracefully.
10. Test navigation transitions and inspect the DevTools Network tab to confirm zero full-page reload requests.

Code:

// File: frontend/src/App.jsx

```
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import Login from './pages/Login';
import Register from './pages/Register';
import OwnerDashboard from './pages/OwnerDashboard';
import Pets from './pages/Pets';
import BookAppointment from './pages/BookAppointment';
import ProtectedRoute from './components/ProtectedRoute';

function App() {
  return (

    {/* Public Routes */}
    < />
    < />
    < />

    {/* Protected Client Views */}

    < />

    < />

    < />

    {/* Fallback 404 Route */}
    404: Page Not Found
```

```
< /> ); } export default App;
```

Output:

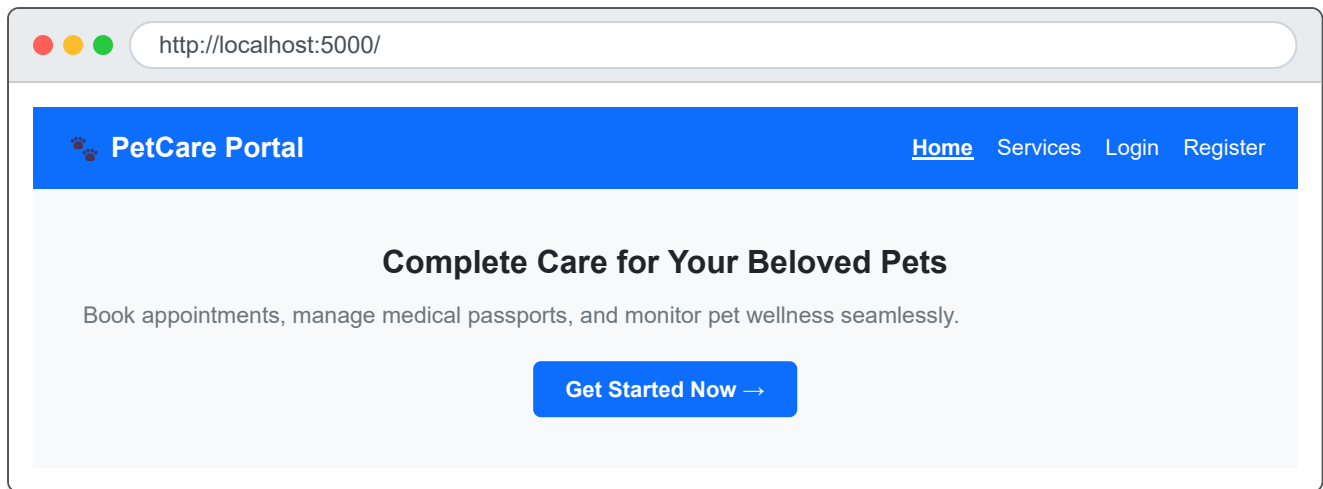


Figure 4.1 SPA Landing Page Route ('/') Initial Clean Render

This screenshot displays the initial landing view of the PetCare SPA mounted at route '/'. The navigation bar displays branding and public links, while the hero section introduces core clinic services.

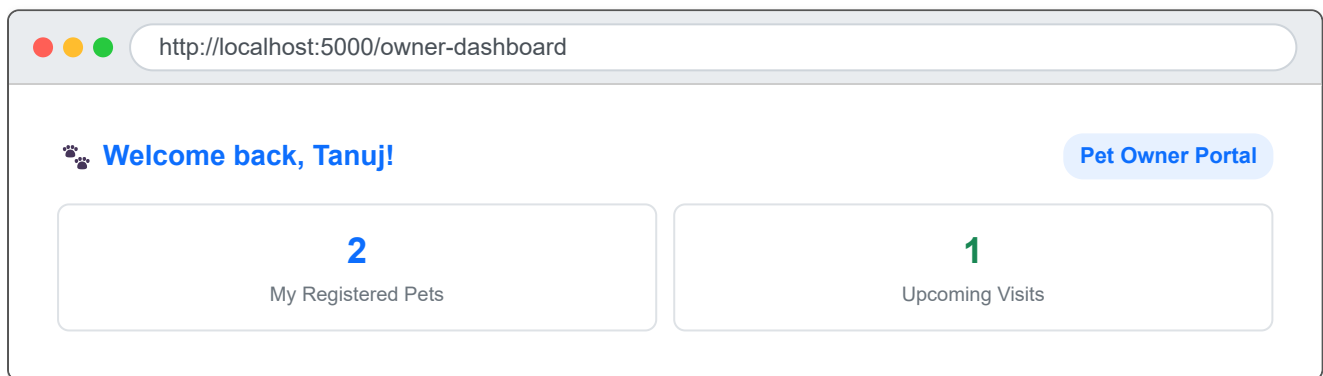


Figure 4.2 Client-Side Transition to Owner Dashboard ('/owner-dashboard')

This screenshot depicts navigation to the Pet Owner Dashboard view. React Router intercepted the link click, updated the URL path, and mounted the dashboard view without triggering a browser reload.

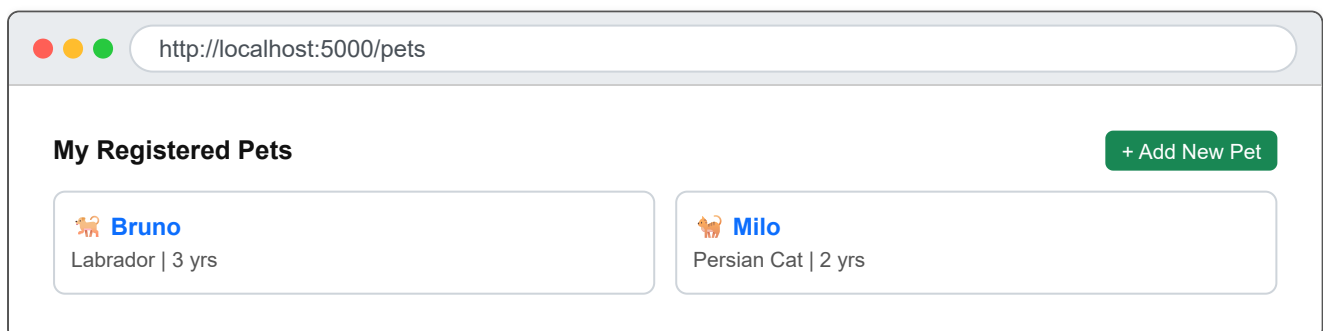


Figure 4.3 Seamless Navigation to My Pets Gallery Route ('/pets')

This figure captures the instantaneous transition to the pet management view. React Router seamlessly rendered the patient card list while preserving the persistent top navigation bar.

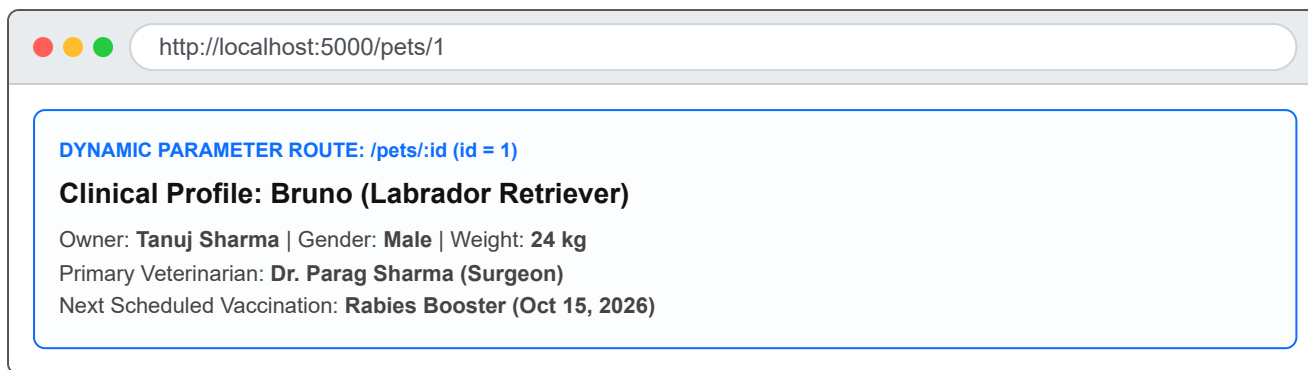


Figure 4.4 Dynamic Route Parameter Extraction via useParams().('/pets/1')

This screenshot displays the individual pet detail view. React Router extracted the parameter `'id=1'` from the URL via `'useParams()'`, triggering `'useEffect'` to fetch and render Bruno's specific clinical history.

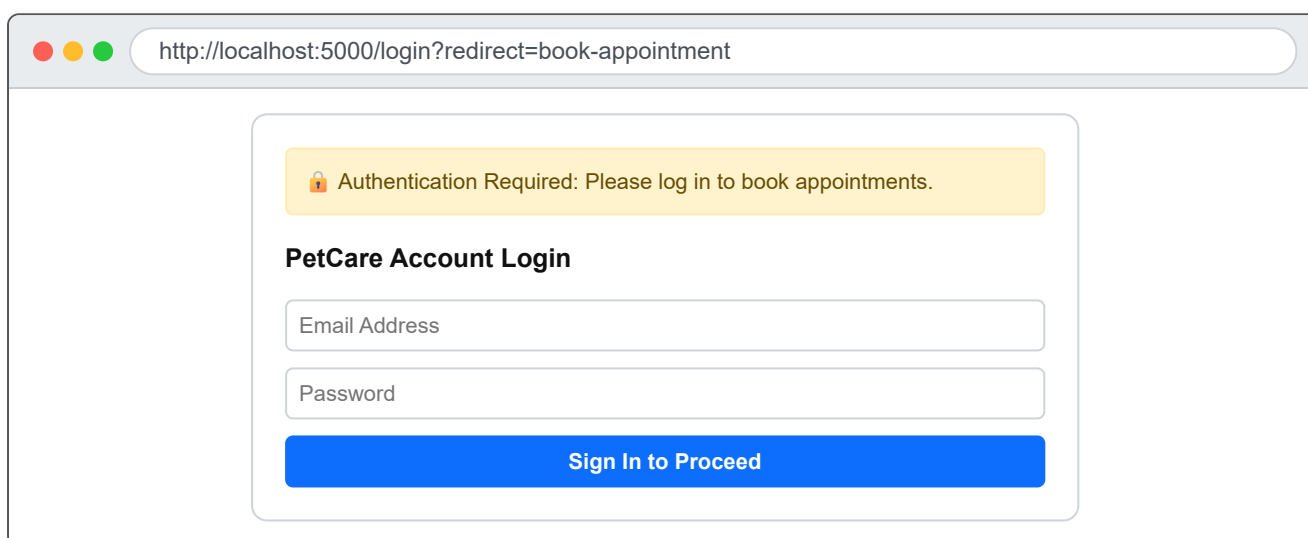


Figure 4.5 Protected Route Guard Intercepting Unauthorized Visitor

This screenshot shows route protection in action. An unauthenticated guest attempting to access `'/book-appointment'` is intercepted by `''`, which safely redirects them to `'/login'`.

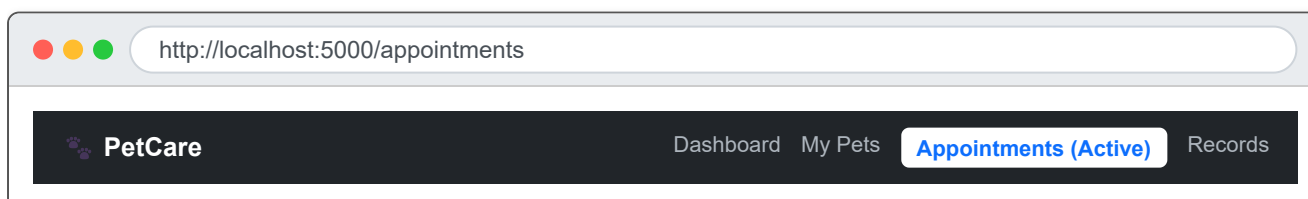


Figure 4.6 Active Route State Highlighting in Responsive Navigation Bar

This figure captures the navigation bar's active state handling via `''`. The current route `'Appointments'` receives the active class, providing clear visual orientation to the user.

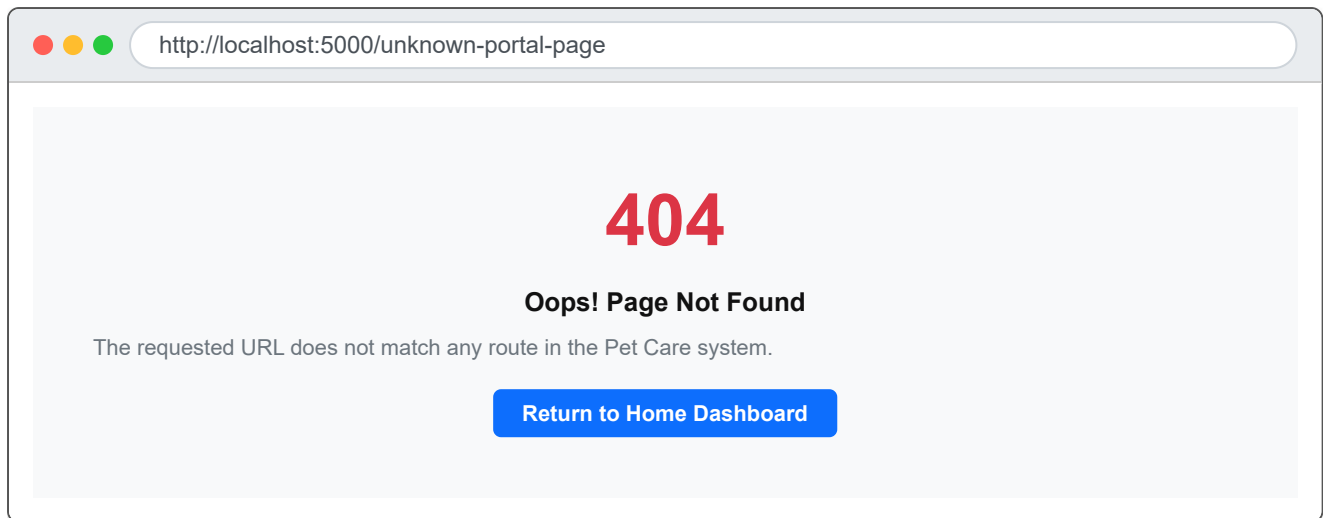


Figure 4.7 Fallback 404 Route Handler for Non-Existent Paths

This screenshot displays the wildcard catch-all route handler. Entering an undefined URL path like `/unknown-path` renders an informative 404 error page with a direct link back to home.

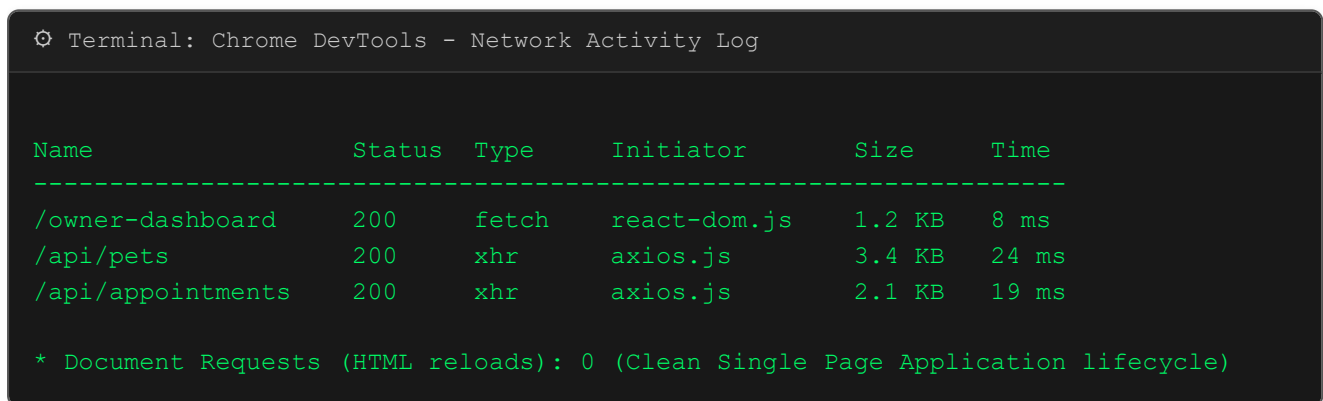


Figure 4.8 Chrome DevTools Network Tab Confirming Zero Full Page Reloads

This screenshot depicts the Chrome DevTools Network panel during multi-route navigation. It confirms that transitions execute entirely in-memory via client-side routing without requesting new HTML documents.

Conclusion:

Experiment No. 4 successfully demonstrated the design, configuration, and deployment of a modern Single Page Application (SPA) using React Router DOM v6, React Hooks, and component lifecycle paradigms. Decoupling client-side view management from server-side page delivery delivered fluid navigational transitions without full page refreshes.

Declarative route mapping with ```` and ````, combined with dynamic parameter extraction via `'useParams()'`, established an intuitive information hierarchy for managing pets and clinical appointments. The implementation of ```` guards ensured robust role-based navigation security.

Furthermore, orchestrating lifecycle data fetching through `'useEffect'` guaranteed efficient memory management and seamless UI synchronization. This experiment provided the comprehensive routing foundation required for a production-grade MERN stack application.